

3주차 실습가이드

포인터 기초부터 시작하는 비트 · 바이트 · 엔디안 · 인코딩

안녕하세요

조교 소개



Minseo Kim (김민서)

융합보안대학원 (2025.09~)

- SW마에스트로 14기 우수자
- 25' 정보보호학회 AI보안공모전 대상
- (주)누리컴퍼니 CEO

imkmsh@khu.ac.kr

010 6345 0815

오늘 실습 순서

- 0단계. 포인터가 뭘지부터 — 주소(&)와 역참조(*)
- 표기법 사전 + 엔디언이란/왜 쓰는가
- 문제 1~4: 문제를 먼저 보여드리고 2~3분 스스로 풀어본 뒤, 같이 풀이를 봅니다
- 1) int를 1바이트씩 들여다보기 2) 문자 하나를 0/1로 (ASCII↔Hexdump 먼저)
- 3) 정수의 엔디언 변환 4) 유니코드 -> UTF-8 인코딩 (코드포인트/U+ 이론 먼저)

포인터 전에: 변수는 메모리 어딘가에 있다

- 변수를 하나 선언하면, 컴퓨터는 메모리에 그 값을 저장할 자리를 정한다
- 그 자리에는 고유한 번지수, 즉 '주소'가 있다
- C에서는 변수 앞에 & 를 붙이면 그 변수의 주소를 구할 수 있다

x
값: 10
주소: 0x1000 (예시)

코드

```
int x = 10;  
printf("%d", x);           // 10 (값)  
printf("%p", (void *)&x); // 0x1000 (주소, 예시)
```

표기법

%p 는 '주소를 출력하라'는 서식 지정자 – void* 타입을 기대하므로 (void*)로 형변환한다

포인터 = 주소를 담는 변수

- 포인터도 변수다. 다만 그 안에 '값' 대신 '다른 변수의 주소'를 담는다
- `int *p;` -> `p`는 `int`를 가리키는 포인터
- `p = &x;` -> `p` 안에 `x`의 주소를 저장 (`p`가 `x`를 '가리킨다')
- `*p` -> `p`가 가리키는 곳의 값을 읽거나 쓴다 (역참조)



코드

```
int x = 10;
int *p = &x; // p에 x의 주소를 저장
*p = 20;     // p가 가리키는 곳(x)에 20을 쓴다
printf("%d", x); // 20 <- p로 x를 바꿨다
```

표기법 미니 사전 — 막히면 여기부터

표기	뜻
<code>0x12</code>	16진수 리터럴. 0x는 '이 숫자는 16진수'라는 표시일 뿐 값의 일부가 아니다. 0x12=10진수 18
<code>%d / %x / %X</code>	printf 서식: 각각 10진수 / 16진수(소문자) / 16진수(대문자)로 출력
<code>%02X</code>	16진수 2자리, 모자라면 앞을 0으로 채워 출력 (예: 5 -> 05)
<code>%p</code>	주소 출력용 서식. void*를 기대하므로 (void *)로 형변환해서 넘긴다
<code>unsigned char</code>	부호 없는 1바이트 정수(0~255). 여기서 '문자'가 아니라 '메모리 1칸' 용도
<code>uint32_t</code>	크기가 정확히 32비트로 고정된 정수 타입

엔디안(Endianness)이란?

- 여러 바이트로 된 값(예: 4바이트 int)을 메모리에 저장할 때, 바이트를 어떤 순서로
- 늘어놓을지 정한 규칙이다. 순서를 정하는 방식이 두 가지 있다:

리틀 엔디안

작은 자리부터
낮은 주소에
-> x86/ARM

빅 엔디안

큰 자리부터
낮은 주소에
-> 네트워크 등

주소(낮음->높음)	리틀 엔디안	빅 엔디안
주소+0	0x78	0x12
주소+1	0x56	0x34
주소+2	0x34	0x56
주소+3	0x12	0x78

정리

비유: '12345678'을 쓸 때 사람은 큰 자리(1)부터 쓴다 -> 빅 엔디안과 비슷한 방식
그런데 x86/ARM 대부분의 컴퓨터는 반대로 '작은 자리부터' 저장하는 리틀 엔디안을 쓴다

왜 리틀 엔디안과 빅 엔디안을 각각 쓸까?

리틀 엔디안을 쓰는 이유

1. 계산은 항상 작은 자리(1의 자리)부터 -> 그 자리가 낮은 주소에 있으면 하드웨어가 더 간단
2. 큰 타입을 작은 타입으로 잘라 읽어도 시작 주소가 그대로 (문제 1과 같은 이유)
3. 인텔 x86이 채택 -> PC/모바일 생태계 대부분이 따름

빅 엔디안을 쓰는 이유

1. 사람이 숫자를 읽는 방식(큰 자리부터)과 같아 디버깅(메모리 덤프)에 직관적
2. 인터넷 표준(TCP/IP)이 '네트워크 바이트 순서'로 채택 -> 기계마다 다른 엔디안 문제 해결
3. 바이트를 그대로 비교해도 숫자 크기 비교와 결과가 같아 정렬/비교에 유리

실습 진행 방법

- practice.c를 열고 이 순서(0단계 -> 문제 1~4)대로 NEED_TO_SOLVE를 채운다
- NEED_TO_SOLVE는 아무 동작도 하지 않는 빈 자리 -> 컴파일은 되지만 결과가 틀림
- 막히면 practice_answer.c와 비교해서 어느 부분이 다른지 찾는다

Windows

```
[CMD]
gcc -Wall -o practice.exe practice.c
practice.exe

[PowerShell]
gcc -Wall -o practice.exe practice.c;
.\practice.exe
```

macOS / Linux

```
gcc -Wall -o practice practice.c
./practice

(macOS는 clang이 gcc로도 동작 -> 별도 설치 없이 바로 됨)
```

규칙

AI 사용 금지. 비트/바이트 변환을 대신 해주는 내장 함수도 사용하지 않는다.

int를 1바이트씩 들여다보기

완성할 코드 (practice.c)

```
void print_bytes_of_int(int value) {
    unsigned char *bytes = (unsigned char *)&value;
    for (int i = 0; i < 4; i++) {
        // TODO: bytes[i]를 "0x12 " 형식으로 출력하세요
    }
}
```

- 이미 unsigned char*로 캐스팅하는 줄은 주어져 있습니다. printf 한 줄만 채우면 됩니다.
- 테스트: print_bytes_of_int(0x12345678); 를 호출하면 무엇이 출력될까요?



먼저 2~3분 동안 혼자 풀어보세요 (힌트만 보고, 풀이는 다음 장에서)

포인터의 '타입'을 바꾸면 읽는 크기가 바뀐다

- int*는 4바이트 정수로, unsigned char*는 1바이트씩 읽으라는 뜻이다
- 주소는 그대로! 타입만 바꾸면 같은 메모리를 다르게 잘라볼 수 있다

코드

```
int value = 0x12345678;
unsigned char *bytes = (unsigned char *)&value;

for (i = 0; i < 4; i++)
    printf("0x%02X ", bytes[i]); // 한 바이트씩
```

왜 0x78 0x56 0x34 0x12 순서일까?

1. 0x12345678 -> 0x12, 0x34, 0x56, 0x78 로 분해
2. 리틀 엔디안: 작은 자리(0x78)를 낮은 주소에 저장
3. bytes[0]=0x78 bytes[1]=0x56
4. bytes[2]=0x34 bytes[3]=0x12

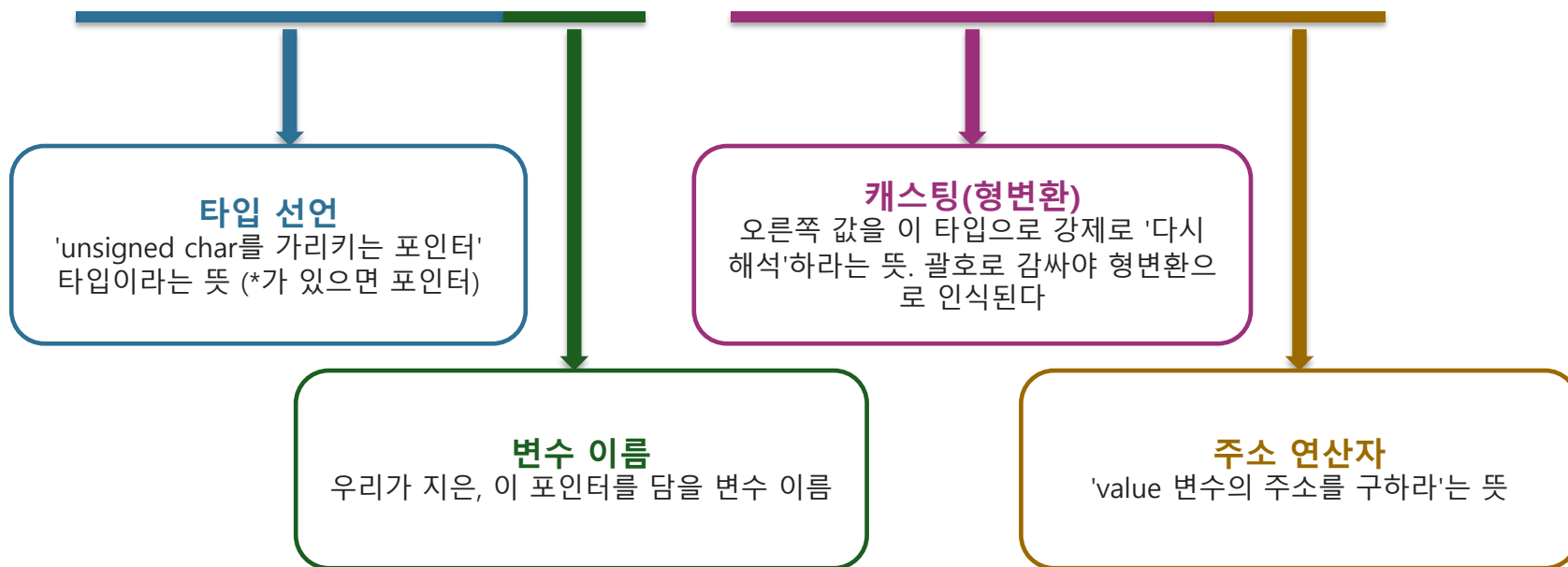
실행 결과 + 표기법

실행 결과: 0x78 0x56 0x34 0x12

표기법: 0x는 '16진수'라는 표시일 뿐 값이 아니다 / %02X는 16진수 2자리, 모자라면 0으로 채움

이 한 줄, 정확히 무슨 뜻일까?

```
unsigned char *bytes = (unsigned char *)&value;
```



한 문장으로

정리: 'value의 주소를 구해서(&value) unsigned char*로 다시 해석한 다음, bytes라는 이름의 포인터 변수에 저장한다' - 이 한 줄은 사실 4개의 작은 동작이 합쳐진 것입니다.

ASCII와 Hex Dump 비교

구분	ASCII	Hex Dump
표현 방식	알파벳, 숫자, 특수문자 등 텍스트	00 ~ FF까지의 16진수 숫자
대상 데이터	일반 텍스트 파일 (.txt, .html 등)	이미지·실행파일·압축파일 등 모든 바이너리 파일
가독성	문자열을 바로 읽을 수 있어 높음	숫자로 되어 있어 바로 읽기 어려움
데이터 손실	문자 범위 밖 데이터는 깨지거나 누락	모든 바이트를 손실 없이 표현

핵심 개념과 실제 활용

ASCII란

컴퓨터의 0/1 데이터를 사람이 읽을 수 있는 문자 128개(영어 대소문자, 숫자, 문장부호 등)에 1:1로 매칭해 둔 표준 규칙.
한계: 메모장으로 이미지(.jpg)나 실행파일(.exe)을 열면 글자가 깨져 보인다 -> ASCII가 표현 못하는 범위의 데이터까지 억지로 문자로 바꾸려다 생기는 현상.

Hex Dump란

파일에 저장된 0/1(바이너리) 데이터를 8비트(1바이트)씩 끊어 16진수(0~F)로 그대로 출력한 것.
장점: 문자든 이미지든 코드든 상관없이 원래 값을 그대로 숫자로 보여주므로 데이터가 깨지거나 왜곡되지 않는다 -> 개발자/보안 전문가가 파일 구조 분석, 디버깅에 사용.

실제 도구(xxd, Hex Editor 등)는 16진수 + ASCII를 나란히 보여준다

```
00000000  48 65 6C 6C 6F 21 0A      Hello!.
```

문자 하나를 0 또는 1로 바꾸기

완성할 코드 (practice.c)

```
int bit_char_to_int(char c) {  
    int result = 0;  
    // TODO: c가 '1'이면 result에 1을 넣으세요  
    return result;  
}
```

- 테스트: `bit_char_to_int('1')` 과 `bit_char_to_int('0')` 은 각각 몇을 반환해야 할까요?
- 힌트: 문자 '1'과 정수 1은 다릅니다 (ASCII에서 '1'은 49번).



먼저 2~3분 동안 혼자 풀어보세요 (힌트만 보고, 풀이는 다음 장에서)

'c == '1' 인지만 확인하면 된다

풀이 코드

```
int bit_char_to_int(char c) {  
    int result = 0;  
    if (c == '1') result = 1;  
    return result;  
}
```

실행 결과 + 왜 이렇게 될까

```
bit_char_to_int('1') = 1  
bit_char_to_int('0') = 0
```

표기법: 문자 리터럴 '1'은 ASCII 코드 49(정수)입니다. 원한다면 '1' - '0' 으로 계산해도 같은 결과(49-48=1)가 나옵니다 – 문자는 결국 작은 정수라는 걸 여기서도 확인할 수 있습니다.

정수의 엔디안(byte order) 변환

완성할 코드 (practice.c)

```
uint32_t swap_endian32(uint32_t value) {  
    unsigned char *bytes = (unsigned char *)&value;  
    // TODO: bytes[0]<->bytes[3], bytes[1]<->bytes[2] 를 교환하세요  
    return value;  
}
```

- 문제 1에서 본 unsigned char* 캐스팅이 그대로 주어져 있습니다.
- 테스트: swap_endian32(0x12345678) 은 얼마가 되어야 할까요?



먼저 2~3분 동안 혼자 풀어보세요 (힌트만 보고, 풀이는 다음 장에서)

양 끝 바이트부터 서로 교환한다

`swap_endian32(0x12345678) -> 0x78563412` (왜?)

1. 메모리 그대로(리틀 엔디안): `[0x78, 0x56, 0x34, 0x12]`
2. `bytes[0]<->bytes[3]` 교환: `[0x12, 0x56, 0x34, 0x78]`
3. `bytes[1]<->bytes[2]` 교환: `[0x12, 0x34, 0x56, 0x78]`
4. 다시 정수로 읽으면(리틀 엔디안): `0x78563412`

실행 결과

```
swap_endian32(0x12345678) -> 0x78563412
```

- 표기법: `uint32_t`는 크기가 정확히 32비트로 '고정된' 정수 타입이다
- 생각해보기: 이 함수를 두 번 연속 적용하면 왜 원래 값으로 돌아올까?

UTF-8

- **UTF-8**은 전 세계의 모든 문자를 컴퓨터에서 일관되게 표현하기 위해 만든 국제 표준 문자 인코딩 방식
"전 세계 모든 글자(한글, 영어, 한자, 아랍어, 이모지 등)를 컴퓨터가 이해할 수 있는 0과 1의 이진수로 바꾸는 전 세계 공통의 규칙"

가변 길이 인코딩 (1~4바이트): 모든 글자를 똑같은 크기로 저장하지 않고, 글자에 따라 크기를 조절

영어/숫자/기호: **1바이트** (메모리 절약)

한글/한자/일본어: **3바이트**

이모지(😄)/고대 문자: **4바이트**

문자 -> 코드포인트 예시

```
'A'   -> 코드포인트 U+0041 (10진수 65)
'가'   -> 코드포인트 U+AC00 (10진수 44032)
'😄'   -> 코드포인트 U+1F600 (10진수 128512)
```

UTF-8 — 접두사 패턴

컴퓨터가 바이트의 앞부분을 읽었을 때 다음과 같이 해석합니다.

0으로 시작하면 (0xxxxxxx):

"아, 이 글자는 딱 1바이트짜리 글자(영어/숫자)구나!" 하고 바로 끝냅니다.

110으로 시작하면 (110xxxxx):

"앞에 1이 2개 있네? 이 글자는 여기서부터 총 2바이트짜리 글자의 시작이구나!"

1110으로 시작하면 (1110xxxx):

"앞에 1이 3개 있네? 이 글자는 여기서부터 총 3바이트짜리 글자(한글 '가' 등)의 시작이구나!"

10으로 시작하면 (10xxxxxx):

"이건 새로운 글자의 시작이 아니라, 앞에서 시작된 글자에 뒤이어 붙는 꼬리(종속) 바이트구나!"

- **글자 크기가 달라도 안전하게 구별:** 1바이트짜리 영어와 3바이트짜리 한글이 무작위로 섞여 있어도, 컴퓨터는 각 바이트의 접두사만 보고 정확하게 잘라서 읽을 수 있습니다.
- **데이터가 유실되어도 자동 복구:** 네트워크 문제로 중간에 데이터가 몇 비트 잘려 나가더라도, 컴퓨터는 다음 데이터에서 1110...이나 0...으로 시작하는 '시작 접두사'를 만나는 순간부터 다시 정상적으로 글자를 읽어 들일 수 있습니다. 중간에 10...으로 시작하는 꼬리 바이트가 나오면 "앞이 잘렸구나" 하고 버리면 되니까요.

비트 연산자: >>, &, |

- >> (오른쪽 시프트): 비트를 오른쪽으로 밀어내며, 밀려난 비트는 버려진다
- & (AND, 마스크): 자리마다 비교해서, 둘 다 1일 때만 그 자리가 1이 된다
- | (OR, 비트합): 자리마다 비교해서, 둘 중 하나라도 1이면 그 자리는 1이 된다

>> : 원하는 상위 비트를 앞으로 밀어오기

```
0xAC00 = 1010110000000000 (16비트)
>>12 = 0000000000001010 = 0xA
      (앞의 4비트만 남고 나머지 12비트는 버려짐)
```

& : 원하는 자리만 남기고 나머지 지우기(마스크)

```
0xD5 = 11010101      0xD5 & 0x0F = 00000101 = 0x05
0x0F = 00001111      (하위 4비트만 남고 나머지는 0으로 지워짐)
```

| : 서로 겹치지 않는 두 조각을 하나로 합치기

```
0xE0 = 11100000 (UTF-8 '3바이트 시작' 표시)      0xE0 | 0x0A = 11101010 = 0xEA
0x0A = 00001010 (실제로 넣고 싶은 값)
```

유니코드 코드포인트 -> UTF-8 인코딩

UTF-8 인코딩 규칙 (이어지는 바이트는 항상 10xxxxxx)

```
codepoint < 0x80      : 0xxxxxxx      (1바이트)
codepoint < 0x800     : 110xxxxx 10xxxxxx (2바이트)
codepoint < 0x10000  : 1110xxxx 10xxxxxx 10xxxxxx (3바이트)
그 이상              : 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx (4바이트)
```

- int encode_utf8(uint32_t codepoint, unsigned char *out) 을 완성하세요.
- out에 인코딩된 바이트를 채우고, 사용한 바이트 개수(1~4)를 반환합니다.
- 테스트: encode_utf8('가' = 0xAC00, out) 은 몇 바이트를, 어떤 값으로 채워야 할까요?



먼저 2~3분 동안 혼자 풀어보세요 (힌트만 보고, 풀이는 다음 장에서)

'가'(U+AC00) -> 0xEA 0xB0 0x80 이 되는 과정

스텝별 풀이

한글 '가'의 유니코드 코드포인트는 0xAC00입니다.

- 반환 바이트 수: 3바이트가 사용됩니다. (0xAC00은 0x800 이상 0x10000 미만 범위에 속함)
- 채워지는 값 (16진수): out 배열에는 순서대로 0xEA, 0xB0, 0x80이 채워집니다.

계산 과정 요약

1. 이진수 변환: 0xAC00 → 1010 1100 0000 0000 (16비트)
2. 3바이트 비트 쪼개기: 뒤에서부터 6비트씩 끊어서 채워 넣습니다.
 - 첫 번째 조각 (상위 4비트): 1010
 - 두 번째 조각 (중간 6비트): 110000
 - 세 번째 조각 (하위 6비트): 000000
3. 접두사 패턴 적용 (1110xxxx 10xxxxxx 10xxxxxx):
 - out[0] = 1110 + 1010 = 11101010 → 0xEA
 - out[1] = 10 + 110000 = 10110000 → 0xB0
 - out[2] = 10 + 000000 = 10000000 → 0x80

실행 결과

'A'(U+0041) 1바이트: 0x41 | '가'(U+AC00) 3바이트: 0xEA 0xB0 0x80 | '😊'(U+1F600) 4바이트: 0xF0 0x9F 0x98 0x80